

The table *InterfaceMethod* of type *PyMethodDef* lists all the public functions. The self-explanatory *PyMethodDef* structure (shown below) is taken from the *methodobject.h* header file in *PYTHON_INCLUDE_PATH/python2.6*. Note that the *hello_world()* function needs no arguments (*METH_NOARGS*), while *add_integers()* and *decode()* take arguments (*METH_VARARGS*).

```
struct PyMethodDef {
    const char *ml_name;      /* The name of the built-in
function/method */
    PyCFunction ml_meth;      /* The C function that implements
it */
    int ml_flags;             /* Combination of METH_xxx flags,
which mostly
                                describe the args expected by
the C func */
    const char *ml_doc;       /* The __doc__ attribute, or NULL
*/
};
typedef struct PyMethodDef PyMethodDef;
```



Note: The *PyCFunction* signature is *PyObject* function(PyObject* self, PyObject* args)*. Cast *ml_meth* if your function signature is different.

Finally, the *PyMODINIT_FUNC* directive has the framework needed for the dynamic loading of the extension to work. It is mandatory to have the initialisation function name in the form *init<module_name>*, where *module_name* is the name of the shared object created. Here, the *initInterfaceMethods()* function is invoked whenever the *InterfaceMethods* module is loaded. As a convention, public method names are started with *<module_name>*, and declared static.

Py_InitModule() exposes the functions described in table *InterfaceMethods*. *Py_InitModule3()* [4] takes an additional doc-string as the third argument. The doc-string is used to summarise the function's behaviour.

Building the module

Let's review the steps required to build the module. There are two methods, each of which we will cover in turn.

Using a Makefile

Here is the Makefile to build the module. Running 'make' builds the *InterfaceMethod.so* shared object. The flag *-shared* *-fPIC* produces a shared object; *-soname* is used to specify the shared object name; and *-Wl,-Bdynamic* informs the linker that the shared object is dynamic.

```
bash>cat Makefile
VERSION=2.6
PYTHON_INCLUDE_PATH=/usr/include/python${VERSION}
```

```
PYTHON_LIB_PATH=/usr/lib/python${VERSION}
all:
    gcc -g -fPIC python_interface.c -o InterfaceMethod.so
-shared -Wl,-soname=InterfaceMethod.so -Wl,-Bdynamic -I${PYTHON_
INCLUDE_PATH} -L${PYTHON_LIB_PATH} -lpython${VERSION}
```

```
clean:
    @rm InterfaceMethod.so 2>/dev/null
bash>make
```

Python distutils

Use the distribution utility [6] from Python, and create *setup.py* to produce the shared object file.

```
bash>cat setup.py
from distutils.core import setup, Extension
```

```
interface_module = Extension('InterfaceMethod', sources =
['python_interface.c'])
setup (name = 'InterfaceMethod',
        version = '1.0',
        description = 'InterfaceMethod demo',
        ext_modules = [interface_module])
```

```
bash>python setup.py build
bash>find build -name "*.so"
```

The *distutils* package is the core for compilation, distribution, and installation of modules. The 'build' command creates the shared object in the build sub-directory.



Note: Use the 'python setup.py install' command (with root privileges) to install the module into *PYTHON_LIB_PATH/python2.6/site-packages*. Use 'python setup.py sdist' to create an archive file under the 'dist' directory, for distribution purposes.

Testing the module from Python

Once the module is built, the next job is to import the module and test it. Make the module visible during Python execution in one of the following ways:

- Install the module into *PYTHON_LIB_PATH/python2.6/site-packages*, as indicated in the note above.
- Export the *PYTHONPATH* variable to which you've added the path to the directory in which your module shared object is, before running Python from the same shell, as follows:

```
bash>export PYTHONPATH=$PYTHONPATH:/path/to/module/
bash>python
```

- Append the path to your module shared object in